NEWBIE 2.0 参赛手册 / Participant Handbook

中英双语版 · 2026-10-20 / Bilingual edition · 2026-10-20

中文 · English

中文

欢迎参加由 **HKUST(GZ) High Performance Computing Association** 组织的 NEWBIE 2.0 新生赛。本次比赛面向本科生，帮助你了解超算赛事、体验高性能计算优化，并选拔参加 ASC26 的选手。

你可以从完整 baseline 开始，理解程序的运行过程，再尝试优化。每次修改后先检查正确性，再比较性能，并保留可以复现成绩的代码和日志。

第一次参赛，请先阅读 QUICKSTART.md：其中包含登录、私人 Conda 环境、pip 换源、共享权重链接、离线缓存和完整运行命令。比赛容器内对应路径为 `/share/QUICKSTART.md`，注意大小写。

1. 从哪里开始

建议按这个顺序完成第一次提交：

1. 使用分配给你的 SSH 私钥登录比赛账号，例如 `team01` 或 `team02`。
2. 阅读 QUICKSTART，检查工具和资源，准备自己的代码目录与环境。
3. 运行一份完整 baseline，确认正确性通过，找到该次运行的 `result.json` 和 `runtime.log`。
4. 在排行榜网站用原登录用户名和队伍 token 登录，选择赛题并上传这两个文件。
5. 在「我的提交」查看校验结果与分项成绩，再开始优化。

SSH 地址为 `10.120.20.219:22000`，排行榜地址为 http://10.120.20.219:8000。SSH 私钥与网站 token 用途不同，请妥善保管。

网站登录始终使用原用户名，如 `team01`、`team02`。网页中的队伍展示名只用于显示，不用于登录；改展示名不会改变你的提交归属和成绩。

2. 共享环境与资源使用约定

你可以使用的环境

比赛环境提供四张 A800 80GB、64 个 CPU 和约 2 TiB 内存，**由所有参赛选手共同使用，不是每队独占一套**。系统工具、赛题原件、模型权重和参考资料也通过公共环境共享。

你的 home 容量上限为 50 GiB，用于自己的代码、Conda 环境、缓存和运行产物。
Python/Conda 环境由各队独立维护，不要安装或修改其他队伍的环境。

`/share` 是只读资料目录。将赛题复制到自己的 home 后修改；复制后的文件如果仍只读，可对自己的副本执行 `chmod -R u+rwX <副本目录>`。模型权重使用共享路径或软链接，不必再下载或复制一份。

运行前先检查，结束后及时释放

不要长时间占卡、闲置占卡或重复启动无用任务。 调试时先用必要的资源确认功能，再进行完整跑分；运行结束或放弃实验时，及时停止自己不再需要的进程，释放 GPU 显存、CPU 和内存。

启动前检查资源是否已被其他选手使用：

```
nvidia-smi
nvidia-smi --query-
gpu=index,name,memory.used,memory.total,utilization.gpu --format=csv
uptime
free -h
```

查看 GPU 显存、利用率和进程列表，同时关注 CPU 负载及可用内存。**GPU 利用率暂时为 0 不代表空闲**，模型可能仍占用大量显存或正在进行 CPU 侧处理。一次检查只是当前快照，也不能保证资源接下来始终空闲。

容器内设备编号以实际枚举为准，可先执行 `nvidia-smi -L`。运行自己的代码时选择合适的可见设备；一键 UnifoLM baseline 默认使用 GPU 0，不会自动寻找空闲卡，外部设置也不能覆盖该助手内部固定的设备选择。若 GPU 0 繁忙，可以等待错峰，或按 QUICKSTART 的自定义启动方式明确选择设备。

错峰运行与相互协调

- 尽量避免在调试高峰同时启动多个完整 benchmark，尤其不要为了占位而申请全部 GPU。
- 耗时较长的实验可以安排在夜间或其他低负载时段。你也可以让自己的 agent 在夜间启动任务，但应让它在启动前再次检查资源；资源繁忙时延后运行，并设置清晰的退出条件，避免无限重试和反复拉起任务。
- 自动任务应保留运行日志，完成后退出；定期检查自己是否留下了重复进程、空转脚本或不再需要的模型实例。使用 agent 仍须遵守同样的资源约定。

- 如果发现持续占用、浪费资源、影响其他选手调试的行为，或无法判断任务归属，可以向赛事主办方沟通。提供发生时间、资源占用截图或必要进程信息，方便协调；不要在公开材料中泄露 token、私钥或他人数据。
- 不要擅自终止其他选手的任务，不要修改他人的文件或环境。资源争议请交由主办方协调。

正式比较性能时，也请记录当时的资源使用情况。其他任务会影响耗时，不能把负载变化误认为自己的优化效果。

3. MPI Attention：优化 CPU 并行计算

本题使用纯 C、OpenMPI 和可选 OpenMP 实现注意力计算：

```
O = softmax(QKᵀ / √D)V
```

输入、输出为 `[B][H][S][D]` 布局的 FP32 数据。三个小项各 40 分，共 120 分。

小项	B	H	S	D	分值
short	48	16	512	64	40
mixed	4	8	2048	128	40
long	1	4	8192	64	40

先跑通 baseline

```
python3 /share/tools/run-mpi-baseline.py
```

该入口使用 `~/competition/mpi-attention-baseline`，默认 16 个 MPI rank、每 rank 一个线程，每项预热 1 次、正式测量 5 次取中位数。结果保存在 `~/benchmark-runs/mpi/<运行时间>/`，结束时打印准确路径。

修改自己的实现后，使用 QUICKSTART 中的自定义命令运行。首次生成 FP64 参考输出时应看到哈希校验通过；跑分时保留 `--ref-dir`。输出与参考的 Frobenius 相对误差必须不超过 **0.01，即 1%**。

可以从哪些方向优化

方向	可以尝试的工作

任务划分	short/mixed 可先考虑按 `(B,H)` 单元分配；long 只有四个这样的单元，可研究 Q 序列行切分或二维切分，改善负载均衡
访存与计算	调整循环顺序、分块、数据打包、缓存复用与 SIMD，减少 K/V 的重复读取
softmax	研究数值稳定的分块或 online softmax，正确处理局部最大值、指数和与输出累加的合并
通信	减少重复传输，尝试通信与计算重叠；最终完整输出的汇总仍须计入耗时
并行布局	比较不同 rank × OpenMP 线程组合、CPU 绑定及 NUMA 亲和，避免超额订阅
精度与编译	在误差阈值内尝试混合精度与编译优化，先确认实际 CPU 支持的指令集

主要可修改的文件是 `src/reference/attention_baseline.c`。你可以新增源文件并在 Makefile 中注册，但必须保持 `attention.h` 的接口语义，并在 rank 0 返回完整输出。

提交前确认

- 三个小项都有对应吞吐、耗时及正确性结论。
- 保留完整运行日志和最终摘要。
- 上传单数文件名 **`result.json`**，与同次运行的 `runtime.log` 配对；不要选底层中间报告 `results.json`。
- 修改后可先运行 `make test` 检查，再执行完整 benchmark。

本题禁止事项

禁止使用 BLAS、MKL、OpenBLAS、Eigen 等外部数学库；禁止修改冻结文件、输入生成、计时或误差判定；禁止绕过参考检查、预计算或回放评测答案、伪造输出和时间，或通过 `LD_PRELOAD` 等方式替换评测行为。冻结文件清单以当前 `manifest.json` 为准。

4. UnifoLM：优化完整视频推理

本题要求在保持输出质量的前提下，降低完整模型推理的运行时间。五个小项各 40 分，共 200 分。

小项	完整交互轮数
`unitree_z1_stackbox`	12

`unitree_z1_dual_arm_stackbox`	7
`unitree_z1_dual_arm_stackbox_v2`	11
`unitree_z1_dual_arm_cleanup_pencils`	8
`unitree_g1_pack_camera`	11

完整运行共 49 轮、103 个视频。每项生成视频与共享 FP32 参考比较，**PSNR 必须严格大于 25 dB**；25 dB 本身不通过。视频集合、帧数及尺寸也必须匹配。

先跑通 FP32 baseline

按 QUICKSTART 安装自己的 UnifoLM 环境，再执行：

```
python3 /share/tools/run-unifolm-baseline.py
```

该入口使用 `~/competition/unifolm-fp32-baseline`，直接读取共享权重和离线缓存。结果保存在 `~/benchmark-runs/unifolm/<运行时间>/`，包括本次 JSON、日志、profile 和视频。

如果修改的是 `~/competition/unifolm-world-model-action`，请使用 QUICKSTART 第 6 节的自定义启动命令，并确认 `PYTHONPATH` 指向这一份源码。**修改一份代码、启动另一份代码，修改不会生效。** 普通 Python 修改通常不需要重装 Conda 环境，但须重新启动进程。

可以从哪些方向优化

方向	可以尝试的工作
性能分析	查看 profile，区分导入、模型加载、编码、推理、同步和视频写入开销，优先优化实际瓶颈
减少重复开销	复用模型、参数及与评测输入无关的资源，减少不必要的分配、复制和同步；保留必需计算
算子实现	分析 attention、卷积、矩阵运算和张量布局，尝试等价融合或更高效的执行实现
精度	尝试混合精度、TF32 等方案，每次重新检查逐项 PSNR，不以速度提升替代质量验证
加载与输出	使用共享 ckpt、离线 CLIP 缓存，优化必要输出的写入路径，同时保留全部规定产物

多 GPU	尝试分项并发或单项内部并行；分项并发可能缩短总等待时间，但不一定降低该项评分耗时

尤其注意 `stackbox_v2` 的精度敏感性，不要假定开启 TF32 后所有小项都能通过。应以自己的完整运行及 PSNR 结果判断。

提交前确认

- 运行完整轮数，保存全部规定视频。
- 每个有效小项有正的 `process_wall_s`、PSNR 和通过结论，日志中有匹配的 `TIMING`、`PSNR` 行。
- 评分使用该项进程耗时，包含导入、模型加载、推理及必需输出写入；纯 CUDA 时间和总运行时间用于诊断，不能替代该项评分时间。PSNR 校验耗时另行记录。
- 精确一致的视频可能产生无穷大 PSNR。由工具自动写成 `psnr_db: null` 与 `psnr_exact_match: true`，日志显示 `inf`；不要自行改成非法 JSON 的 `Infinity`。
- 某项执行失败、缺视频或视频损坏时，查看该项原因。健康项可保留成绩；不要为了显示成功而手改 JSON 或日志。

本题禁止事项

禁止减少规定轮数、采样步数、必需帧数或尺寸；禁止改变固定输入与种子、替换成另一模型、回放或复制参考视频、跳过必需计算、修改计时范围或质量阈值、隐藏错误和伪造 PSNR。短功能测试只供本地排查，不能作为完整成绩上传。

5. 如何提交与查看成绩

上传同一次运行的两个文件

1. 使用原用户名和队伍 token 登录排行榜。
2. 打开「提交」，选择对应赛题。
3. 上传 **`result.json`** + **`runtime.log`**，各不超过 5 MiB，使用 UTF-8 原始文本。
4. 在「我的提交」查看校验结果、分项成绩和日志。

日常成绩提交无需 Git URL 或 Git SHA。不要上传视频、权重、压缩包、单项诊断 JSON，也不要混用不同运行的 JSON 和日志。提交后请看页面的校验结论，上传完成并不代表成绩已通过。

分数怎么算

```
MPI 每项 = 40 × 本队该项有效 GF/s / 全场该项最高有效 GF/s
UnifoLM 每项 = 40 × 全场该项最短有效耗时 / 本队该项有效耗时
总分 = MPI 120 分 + UnifoLM 200 分 = 320 分
```

每队每个小项分别取历史最好有效成绩，不要求所有最优小项来自同一次提交。某次某项失败，只影响那次该项，不会抹去此前有效成绩。其他队伍刷新最快值时，你的百分比分数也可能变化。

如果某次提交不希望参与计分，可以在「我的提交」中选择「屏蔽计分」；之后也可恢复。请注意：屏蔽会影响该次提交的所有解析记录，榜单会回退到你的其他有效成绩，原始材料仍保留。

如果显示「部分通过」，查看哪些小项有效、哪些需要修复。如果显示「等待复核」，按提示与主办方沟通，不要通过删除标记或改写材料来绕过复核。停止接收提交的时间和最终成绩确认安排，以赛事通知为准。

6. 每次优化建议保留什么

为避免“跑出了快成绩却找不到对应代码”，建议每次保留：

- 本次源码版本或完整代码备份。
- 编译参数、环境版本，以及 rank/线程/GPU 使用方式。
- 原始 `result.json`、`runtime.log` 和分项 profile。
- 必要的视频或计算输出，以及当时的资源占用记录。
- 与 baseline 的分项对比和对正确性结果的说明。

每次使用独立结果目录。确认不再需要复现或核验后，再清理旧产物；避免把共享权重复制进自己的 50 GiB home。

7. 获奖队伍须提交代码和报告

获奖候选队伍必须提交可复现源码和技术报告，并配合成绩复核，完成后确认奖项。 具体交付渠道、截止日期及复跑安排，以主办方通知为准。

源码材料

请提交实际产生有效成绩的代码、构建与运行脚本、依赖版本或环境锁定文件、编译参数及资源布局，并提供一份从新环境复现的说明。

如果不同小项的历史最优来自不同版本，需要逐项说明对应代码版本、网站提交 ID 和解析记录。不要只交最后一次代码，却无法复现之前计分的小项。

同时保留并按要求提供对应的原始结果、完整日志、profile 和必要输出。共享权重及公共参考通过路径说明，不重复打包；材料中不包含 SSH 私钥、队伍 token 或其他队伍的数据。

技术报告

报告应说明：

1. baseline 的运行结果与主要瓶颈。
2. 你做了哪些算法或实现优化，为什么有效。
3. 使用的精度、算子替换和并行方法。
4. 逐小项性能、加速比及正确性验证结果。
5. 计时口径、环境和资源配置，以及如何复现。
6. 已知限制、性能波动和未通过的尝试。

最终成绩需能通过提交的代码和说明复现。日常上传阶段只需成对 JSON/log，但请从参赛开始就保留源码和实验记录，以便获奖复核。

8. 已有旧版代码时如何更新

首次复制当前共享赛题的选手，可以直接按 QUICKSTART 开始。若你已有旧版副本，先备份自己的改动，再更新公共评测文件。

MPI 可使用以下命令，`--repo` 按自己的实际目录替换：

```
python3 /share/tools/update-mpi-release.py --repo ~/competition/mpi-attention-baseline
cd ~/competition/mpi-attention-baseline
make test
```

升级工具保留你的实现、Makefile 和运行参数。如果提示冻结文件不匹配，先检查差异或联系主办方，不要直接覆盖 manifest 来消除提示。

UnifoLM 已有副本需要同步当前共享赛题中的 `scripts/run_compute_benchmark.sh`，以便一项失败后继续其他独立项，并保留健康项的质量证据。如果你改过自己的启动脚本，请先备份并合并更新，避免覆盖自定义设置。

更多运行与排查方法见 QUICKSTART.md；两题的具体提交约束见 `/share/problems/<赛题>/SUBMISSION.md`，视频质量细节见 `/share/problems/unifolm-world-model-action/PSNR.txt`。

English

Welcome to NEWBIE 2.0, the newcomer competition organized by **HKUST(GZ) High Performance Computing Association**. This undergraduate competition introduces supercomputing contests, offers hands-on experience in high-performance computing optimization, and helps select participants for ASC26.

Start with a complete baseline, understand how the program works, and then experiment with optimizations. After every change, check correctness before comparing performance, and keep the code and logs needed to reproduce your results.

First-time participants should read QUICKSTART.md first. It covers login, private Conda environments, the pip mirror, links to shared weights, offline caching, and complete run commands. Inside the competition container, the file is `/share/QUICKSTART.md`; the filename is case-sensitive.

1. Getting started

Recommended steps for your first submission:

1. Log in to your assigned account, such as `team01` or `team02`, using your SSH private key.
2. Read QUICKSTART, inspect available tools and resources, and prepare your own source directory and environment.
3. Run a complete baseline, check correctness, and find that run's `result.json` and `runtime.log`.
4. Log in to the leaderboard using your original account name and team token, select the problem, and upload both files.
5. Check validation and per-case scores under “我的提交” (My Submissions), then begin optimizing.

SSH: `10.120.20.219:22000`. Leaderboard: http://10.120.20.219:8000. Your SSH private key and website token have different purposes; protect both.

Always log in using your original account name, such as `team01` or `team02`. A team display name is only for presentation and cannot be used to log in. Changing it does not change ownership of your submissions or scores.

2. Shared environment and resource etiquette

Available environment

The competition provides four A800 80GB GPUs, 64 CPUs, and approximately 2 TiB of RAM. **All participants share these resources; they are not reserved separately for each team.**

System tools, original problem packages, model weights, and reference materials are also shared through the common environment.

Your home has a 50 GiB capacity limit for your code, Conda environments, caches, and artifacts. Each team maintains its own Python/Conda environment. Do not install into or change another team's environment.

`/share` is read-only. Copy the problem source into your home before editing. If copied files are still read-only, run `chmod -R u+rwX <your-copy>` on your own copy. Read weights through shared paths or symbolic links instead of downloading or duplicating them.

Check before running; release resources afterward

Do not monopolize GPUs for long periods, hold GPUs while idle, or launch unnecessary duplicate jobs. Use only the resources needed for initial debugging, then run the full benchmark. When a run finishes or you abandon an experiment, stop your own unneeded processes and release GPU memory, CPU, and RAM.

Before starting, check whether other participants are using the resources:

```
nvidia-smi
nvidia-smi --query-
gpu=index,name,memory.used,memory.total,utilization.gpu --format=csv
uptime
free -h
```

Inspect GPU memory, utilization, and the process list, as well as CPU load and available RAM. **Zero GPU utilization does not necessarily mean a GPU is free:** a model may still occupy substantial memory or be performing CPU-side work. One check is only a snapshot, not a guarantee that resources will remain available.

Use the actual device numbering inside the container; inspect it with `nvidia-smi -L`. Select suitable visible devices for your own code. The one-command UnifoLM baseline fixes its device to GPU 0 and does not automatically select an idle GPU; an external environment setting cannot override that helper's internal choice. If GPU 0 is busy, wait for a quieter period or use QUICKSTART's custom launch method to choose a device explicitly.

Run off-peak and coordinate with others

- Avoid launching several full benchmarks during peak debugging hours. In particular, do not claim all GPUs merely to reserve them.

- Schedule long experiments at night or during other low-load periods. You may ask your own agent to start a task overnight, but it should check resource availability again immediately before launch. If resources are busy, defer the run and use clear stopping conditions to avoid endless retries or repeated restarts.
- Automated jobs must keep logs and exit when complete. Regularly check for duplicate processes, idle loops, and model instances you no longer need. Agents must follow the same resource rules as human participants.
- If sustained occupation, resource waste, or interference affects your debugging—or you cannot identify the owner of a job—contact the event organizers. Provide the time, resource-usage screenshots, or necessary process details to help coordination. Do not expose tokens, private keys, or other participants' data in public materials.
- Do not terminate another participant's job or change their files or environment. Ask the organizers to resolve resource disputes.

Record resource usage when making performance comparisons. Other jobs affect timing; do not mistake changes in background load for gains from your optimization.

3. MPI Attention: optimize CPU parallel computation

Implement attention using pure C, OpenMPI, and optional OpenMP:

```
O = softmax(QKᵀ / √D)V
```

Input and output are FP32 tensors in `[B][H][S][D]` layout. There are three cases, worth 40 points each and 120 points in total.

Case	B	H	S	D	Points
short	48	16	512	64	40
mixed	4	8	2048	128	40
long	1	4	8192	64	40

Run the baseline first

```
python3 /share/tools/run-mpi-baseline.py
```

The helper uses `~/competition/mpi-attention-baseline`. Defaults are 16 MPI ranks, one thread per rank, one warm-up, and five timed repetitions per case, using the median. Results are stored in `~/benchmark-runs/mpi/<run-id>/`; exact paths are printed at completion.

After editing your implementation, use the custom commands in QUICKSTART. When generating FP64 references for the first time, confirm that their hashes pass validation. Keep `--ref-dir` during benchmarking. The relative Frobenius error against the reference must be no greater than **0.01, or 1%**.

Optimization ideas

Direction	Possible work
Task partitioning	Start with `(B,H)` units for short/mixed. Long has only four such units; consider partitioning Q rows or using a two-dimensional partition to improve load balance
Memory and computation	Adjust loop order, tiling, packing, cache reuse, and SIMD to reduce repeated reads of K/V
Softmax	Explore numerically stable blocked or online softmax, correctly combining local maxima, exponential sums, and output accumulations
Communication	Reduce redundant transfers and overlap communication with computation; collecting the final complete output must remain inside timing
Parallel layout	Compare rank × OpenMP thread combinations, CPU binding, and NUMA affinity without oversubscription
Precision and compilation	Experiment with mixed precision and compiler optimization within the error limit; first check the actual CPU instruction set

The main editable file is `src/reference/attention_baseline.c`. You may add source files and register them in the Makefile, but must preserve the `attention.h` interface semantics and return a complete output on rank 0.

Before submitting

- Include throughput, timing, and a correctness outcome for each of the three cases.

- Keep the complete execution log and final summary.
- Upload the singular **`result.json`** with `runtime.log` from the same run, rather than the intermediate raw `results.json`.
- After making changes, you can run `make test` before the complete benchmark.

Prohibited actions

Do not use external mathematical libraries such as BLAS, MKL, OpenBLAS, or Eigen. Do not change frozen files, input generation, timing, or error checks; bypass reference verification; precompute or replay benchmark answers; fabricate outputs or timings; or replace evaluation behavior through mechanisms such as `LD_PRELOAD`. The current `manifest.json` defines the frozen-file list.

4. UnifoLM: optimize complete video inference

Reduce runtime while preserving the quality of complete model inference. Five cases are worth 40 points each, for 200 points in total.

Case	Required interaction rounds
`unitree_z1_stackbox`	12
`unitree_z1_dual_arm_stackbox`	7
`unitree_z1_dual_arm_stackbox_v2`	11
`unitree_z1_dual_arm_cleanup_pencils`	8
`unitree_g1_pack_camera`	11

The complete workload has 49 rounds and 103 videos. Each case is compared with the shared FP32 reference, and **PSNR must be strictly greater than 25 dB**; exactly 25 dB fails. Video sets, frame counts, and dimensions must also match.

Run the FP32 baseline first

Install your private UnifoLM environment using QUICKSTART, then run:

```
python3 /share/tools/run-unifolm-baseline.py
```

The helper uses `~/competition/unifolm-fp32-baseline`, reading shared weights and offline caches directly. Results go to `~/benchmark-runs/unifolm/<run-id>/`, including

JSON, log, profiles, and videos.

If your changes are in `~/competition/unifolm-world-model-action`, use the custom launch command in QUICKSTART Section 6 and ensure that `PYTHONPATH` points to that source copy. **Editing one copy and launching another will not apply your changes.** Ordinary Python edits usually do not require reinstalling Conda, but you must restart the process.

Optimization ideas

Direction	Possible work
Profiling	Separate imports, model loading, encoding, inference, synchronization, and video writes; prioritize measured bottlenecks
Repeated overhead	Reuse models, parameters, and resources independent of evaluation inputs; reduce unnecessary allocation, copies, and synchronization while retaining required computation
Operators	Analyze attention, convolution, matrix operations, and tensor layouts; try equivalent fusion or more efficient implementations
Precision	Experiment with mixed precision or TF32 and recheck per-case PSNR every time; speed does not replace quality validation
Loading and output	Use shared checkpoints and offline CLIP caching; optimize required writes while retaining all prescribed artifacts
Multiple GPUs	Try concurrent cases or parallelism within one case. Concurrent cases may reduce overall waiting time without reducing the time used to score an individual case

Be particularly careful with the precision sensitivity of `stackbox_v2`. Enabling TF32 does not guarantee that all cases pass. Judge your implementation using your own complete runs and PSNR results.

Before submitting

- Complete every required round and save all prescribed videos.
- Every scoring case needs a positive `process_wall_s`, PSNR, and passing outcome, with corresponding `TIMING` and `PSNR` lines in the log.

- Scoring uses per-case process time, including imports, model loading, inference, and required output writes. Pure CUDA time and overall runtime are diagnostic and cannot replace that case's scoring time. PSNR validation time is recorded separately.
- Exact matches may yield infinite PSNR. The tool represents this automatically as `psnr_db: null` and `psnr_exact_match: true`, with `inf` in the log. Do not manually insert the invalid JSON value `Infinity`.
- If a case fails execution, has missing videos, or contains corrupt videos, inspect its reason. Healthy cases can still score; do not edit JSON or logs to make a failed run appear successful.

Prohibited actions

Do not reduce required rounds, sampling steps, frame counts, or dimensions; change fixed inputs or seeds; substitute a different model; replay or copy reference videos; skip required computation; change timing boundaries or quality thresholds; conceal errors; or fabricate PSNR. Short functional tests are for local debugging only and cannot be uploaded as complete results.

5. Submit and check scores

Upload two files from the same run

1. Log in using your original account name and team token.
2. Open “提交” (Submit) and select the problem.
3. Upload **`result.json`** + **`runtime.log`** as original UTF-8 text, at most 5 MiB each.
4. Review validation, case scores, and logs under “我的提交” (My Submissions).

Routine submissions require neither a Git URL nor a Git SHA. Do not upload videos, weights, archives, single-case diagnostic JSON, or materials mixed from different runs. Check the validation outcome after upload: a completed upload does not mean the result is accepted.

How scoring works

```
MPI case points = 40 × your team's valid GF/s / highest valid GF/s for
that case
UnifoLM case points = 40 × shortest valid time for that case / your
team's valid time
Total = MPI 120 + UnifoLM 200 = 320 points
```

Each team's cases independently use their best valid historical results; they need not come from one submission. A failed case affects only that case in that submission, without erasing earlier valid results. Your percentage scores may change when another team sets a new fastest result.

To exclude a submission from scoring, choose “屏蔽计分” (Exclude from Scoring) under My Submissions. You can restore it later. Exclusion applies to every parsing attempt of that submission; the leaderboard falls back to your other valid results, while original materials remain available.

If you see “部分通过” (Partially Accepted), inspect which cases passed and which need fixing. If you see “等待复核” (Pending Review), follow the message and contact the organizers. Do not delete flags or rewrite materials to bypass review. Submission closing times and final result confirmation follow the event announcements.

6. What to keep after each optimization

To avoid obtaining a fast result without knowing which code produced it, keep:

- The source version or a complete source backup for the run.
- Compiler flags, environment versions, and rank/thread/GPU choices.
- Original `result.json`, `runtime.log`, and per-case profiles.
- Necessary videos or computation outputs, plus a record of resource usage during the run.
- Per-case comparisons against the baseline and an explanation of correctness results.

Use a separate results directory for every run. Delete old artifacts only when they are no longer needed for reproduction or verification. Do not copy shared weights into your 50 GiB home.

7. Award-winning teams must submit code and a report

Teams shortlisted for awards must provide reproducible source code and a technical report, and cooperate with result verification before awards are confirmed. Delivery channels, deadlines, and rerun arrangements follow the organizers' announcements.

Source materials

Submit the actual code that produced your valid results, build and run scripts, dependency versions or an environment lock file, compiler flags, and resource layout. Include instructions for reproducing results in a fresh environment.

If your best historical cases come from different versions, identify the code version, website submission ID, and parsing record for each case. Do not submit only your latest code if it cannot reproduce earlier scoring cases.

Retain and provide the original results, complete logs, profiles, and necessary outputs as requested. Reference shared weights and public reference data by path rather than packaging duplicate copies. Do not include SSH private keys, team tokens, or another team's data.

Technical report

Explain:

1. Your baseline results and principal bottlenecks.
2. The algorithmic or implementation changes you made and why they help.
3. Precision choices, operator replacements, and parallel methods.
4. Per-case performance, speedups, and correctness validation.
5. Timing boundaries, environment, resource configuration, and reproduction steps.
6. Known limitations, performance variation, and unsuccessful experiments.

Final results must be reproducible using your submitted code and instructions. Routine uploads require only paired JSON/log files, but preserve source and experiment records from the beginning so that award verification is possible.

8. Updating an existing source copy

If you are copying the current shared problems for the first time, follow QUICKSTART directly. If you have an older copy, back up your changes before updating the common evaluation files.

For MPI, adjust `--repo` to your actual repository path:

```
python3 /share/tools/update-mpi-release.py --repo ~/competition/mpi-attention-baseline
cd ~/competition/mpi-attention-baseline
make test
```

The updater preserves your implementation, Makefile, and run parameters. If it reports frozen-file differences, inspect them or contact the organizers. Do not overwrite the manifest merely to suppress the warning.

Existing UnifoLM copies need the current shared `scripts/run_compute_benchmark.sh` to continue other independent cases after a failure and preserve healthy correctness evidence. If you modified your launcher, back it up and merge the update instead of overwriting your custom settings.

For more commands and troubleshooting, see QUICKSTART.md. Detailed submission requirements are in `/share/problems/<problem>/SUBMISSION.md`; video-quality details are in `/share/problems/unifolm-world-model-action/PSNR.txt`.